

LAPORAN UAS STRUKTUR DATA

Implementasi DSS Penentuan Rute Kurir (Graph + Dijkstra)



INSTIKI
INSTITUT BISNIS DAN
TEKNOLOGI INDONESIA

**Dosen Pengampu Mata Kuliah Struktur Data:
(I Made Gede Sri Artha, S.T.)**

Oleh:

Ni Made Era Purnama Dewi (2501010007)

Dewa Ayu Sarika Citra (2501010134)

INSTITUT BISNIS DAN TEKNOLOGI INDONESIA

2025/2026

BAB I

PENDAHULUAN

A. Latar belakang

Perkembangan bisnis jasa pengiriman barang yang semakin pesat menuntut perusahaan logistik untuk memberikan layanan yang cepat, tepat, dan efisien. Salah satu faktor yang memengaruhi kualitas layanan pengiriman adalah kemampuan kurir dalam menentukan rute perjalanan yang optimal. Dalam praktiknya, kurir sering kali harus mengantarkan paket ke beberapa lokasi berbeda dalam satu perjalanan. Penentuan rute yang kurang tepat dapat menyebabkan waktu tempuh menjadi lebih lama, biaya operasional meningkat, serta keterlambatan pengiriman kepada pelanggan.

Permasalahan tersebut dapat diselesaikan dengan memanfaatkan teknologi Decision Support System (DSS) yang mampu membantu pengambilan keputusan berdasarkan data yang tersedia. Pada kasus penentuan rute kurir, hubungan antar lokasi pengiriman dapat direpresentasikan dalam bentuk graph, di mana lokasi direpresentasikan sebagai node (vertex) dan jarak antar lokasi direpresentasikan sebagai edge yang memiliki bobot.

Salah satu algoritma graph yang dapat digunakan untuk mencari jalur terpendek adalah algoritma Dijkstra. Algoritma ini mampu menentukan jarak minimum dari suatu titik awal menuju titik tujuan berdasarkan bobot yang dimiliki setiap edge. Dengan menerapkan algoritma Dijkstra pada sistem pendukung keputusan, proses penentuan rute pengiriman dapat dilakukan secara lebih efektif dan efisien.

Berdasarkan permasalahan tersebut, dibuatlah sebuah Decision Support System (DSS) Penentuan Rute Kurir Menggunakan Graph dan Algoritma Dijkstra yang dapat membantu kurir menentukan urutan pengiriman paket dengan jarak tempuh yang lebih optimal.

B. Rumusan masalah

1. Bagaimana memodelkan permasalahan rute pengiriman kurir menggunakan struktur data graph?
2. Bagaimana menerapkan algoritma Dijkstra untuk menentukan jalur terpendek dalam proses pengiriman paket?
3. Bagaimana membangun sistem pendukung keputusan yang dapat membantu kurir menentukan rute pengiriman secara efisien?
4. Bagaimana menampilkan hasil perhitungan rute dalam bentuk visualisasi graph dan peta interaktif?

C. Tujuan

1. Merancang model graph untuk merepresentasikan hubungan antar lokasi pengiriman.
2. Mengimplementasikan algoritma Dijkstra dalam pencarian jalur terpendek.
3. Membangun Decision Support System (DSS) yang dapat membantu menentukan rute pengiriman paket.
4. Menampilkan hasil rekomendasi rute beserta visualisasi graph dan peta lokasi pengiriman.

5. Menganalisis efektivitas penggunaan graph dalam proses pengambilan keputusan pada sistem pengiriman barang.

D. Manfaat

Penelitian ini diharapkan dapat memberikan manfaat dalam membantu proses pengambilan keputusan penentuan rute pengiriman paket yang lebih efektif dan efisien melalui penerapan struktur data graph dan algoritma Dijkstra, sehingga dapat mengoptimalkan jarak tempuh, waktu pengiriman, serta meningkatkan pemahaman mengenai implementasi graph pada permasalahan dunia nyata di bidang logistik dan distribusi barang.

BAB II

DASAR TEORI

A. Struktur data graph

Graph merupakan salah satu struktur data yang digunakan untuk merepresentasikan hubungan antar objek. Graph terdiri dari sekumpulan simpul (vertex atau node) dan sisi (edge) yang menghubungkan antar simpul. Struktur data graph banyak digunakan untuk memodelkan berbagai permasalahan dunia nyata seperti jaringan komputer, peta jalan, media sosial, dan sistem transportasi.

Secara umum, graph dapat dibedakan menjadi beberapa jenis, antara lain directed graph (graf berarah), undirected graph (graf tidak berarah), dan weighted graph (graf berbobot). Pada penelitian ini digunakan weighted undirected graph karena setiap hubungan antar lokasi memiliki bobot berupa jarak tempuh dan dapat dilalui dua arah.

Dalam sistem penentuan rute kurir, lokasi gudang dan pelanggan direpresentasikan sebagai node, sedangkan jarak antar lokasi direpresentasikan sebagai edge yang memiliki bobot tertentu. Representasi graph yang digunakan adalah adjacency list dan adjacency matrix untuk menunjukkan hubungan antar node secara lebih terstruktur.

B. Weighted Undirected Graph

Weighted Undirected Graph adalah graph yang memiliki bobot pada setiap edge dan tidak memiliki arah hubungan. Bobot biasanya berupa jarak, biaya, atau waktu tempuh. Pada sistem penentuan rute kurir, bobot edge berupa jarak antar lokasi dalam satuan kilometer sehingga graph dapat digunakan untuk menghitung jalur terpendek.

C. Adjacency List

Adjacency List merupakan salah satu metode representasi graph yang menyimpan daftar node yang terhubung dengan setiap node. Metode ini lebih hemat memori karena hanya menyimpan hubungan yang benar-benar ada pada graph. Pada sistem ini, Adjacency List digunakan untuk menampilkan keterhubungan antar lokasi pengiriman.

D. Adjacency Matrix

Adjacency Matrix adalah representasi graph dalam bentuk matriks atau tabel. Baris dan kolom merepresentasikan node, sedangkan nilai pada perpotongan baris dan kolom menunjukkan hubungan atau bobot antar node. Pada sistem ini, Adjacency Matrix digunakan untuk menampilkan hubungan antar lokasi beserta jaraknya.

E. Geodesic Distance

Geodesic Distance merupakan metode perhitungan jarak berdasarkan koordinat geografis berupa latitude dan longitude. Metode ini menghasilkan jarak yang lebih akurat karena memperhitungkan bentuk permukaan bumi. Pada sistem ini, Geodesic Distance digunakan untuk menghitung jarak antar lokasi pelanggan.

F. Streamlit

Streamlit merupakan framework Python yang digunakan untuk membangun aplikasi web interaktif dengan cepat dan sederhana. Pada penelitian ini, Streamlit digunakan sebagai antarmuka pengguna untuk memasukkan data pelanggan dan menampilkan hasil rekomendasi rute.

G. Python

Python adalah bahasa pemrograman tingkat tinggi yang banyak digunakan dalam pengembangan aplikasi, analisis data, kecerdasan buatan, dan sistem berbasis web. Python dipilih karena memiliki sintaks yang sederhana dan didukung oleh banyak library yang memudahkan proses pengembangan system.

H. NetworkX

NetworkX adalah library Python yang digunakan untuk membuat, mengelola, dan menganalisis graph. Library ini menyediakan berbagai algoritma graph, termasuk algoritma Dijkstra yang digunakan dalam penelitian ini.

I. OpenStreetMap dan Folium

OpenStreetMap merupakan layanan peta digital berbasis data terbuka yang dapat digunakan untuk menampilkan lokasi geografis. Folium adalah library Python yang digunakan untuk mengintegrasikan peta OpenStreetMap ke dalam aplikasi. Pada sistem ini, OpenStreetMap dan Folium digunakan untuk menampilkan lokasi pelanggan dan rute pengiriman.

J. Decision Support System (DSS)

Decision Support System (DSS) merupakan sistem informasi interaktif yang menyediakan informasi, pemodelan, dan manipulasi data untuk membantu proses pengambilan keputusan. DSS digunakan dalam situasi yang semi terstruktur maupun tidak terstruktur, di mana pengambil keputusan tidak selalu mengetahui secara pasti solusi terbaik yang harus dipilih. Menurut Kusri (dalam Zulkifli & Sariffudin, 2017:69), DSS berfungsi sebagai alat bantu yang mendukung proses pengambilan keputusan melalui pengolahan data dan penyajian informasi yang relevan.

DSS tidak mengambil alih keputusan pengguna, melainkan mengolah data yang tersedia menggunakan metode tertentu untuk menghasilkan rekomendasi yang dapat dijadikan bahan pertimbangan. Pada penelitian ini, data yang digunakan berupa koordinat lokasi dan jarak antar titik pengiriman yang kemudian diproses menggunakan struktur data graph dan algoritma Dijkstra. Hasil pengolahan tersebut berupa rekomendasi urutan rute pengiriman yang paling efisien berdasarkan jarak terpendek.

Dalam sistem yang dikembangkan, DSS berperan membantu kurir menentukan urutan lokasi yang harus dikunjungi agar proses pengiriman paket dapat diselesaikan dengan jarak tempuh yang lebih pendek dan waktu yang lebih efisien. Dengan demikian, DSS dapat

membantu meningkatkan efektivitas proses distribusi dan mendukung pengambilan keputusan dalam penentuan rute pengiriman.

K. Algoritma Dijkstra

Algoritma Dijkstra merupakan salah satu algoritma pencarian jalur terpendek (*shortest path*) pada graph berbobot yang diperkenalkan oleh Edsger W. Dijkstra pada tahun 1959 (Dijkstra, 1959). Algoritma ini digunakan untuk mencari jarak minimum dari satu node sumber menuju node tujuan dengan mempertimbangkan bobot pada setiap edge.

Prinsip kerja algoritma Dijkstra adalah memilih node dengan jarak terkecil yang belum dikunjungi, kemudian memperbarui jarak menuju node-node yang terhubung dengannya. Proses tersebut dilakukan secara berulang hingga seluruh node telah dievaluasi atau tujuan telah ditemukan.

Pada sistem DSS penentuan rute kurir, algoritma Dijkstra digunakan untuk menghitung jarak terpendek dari lokasi kurir saat ini menuju pelanggan berikutnya. Hasil perhitungan tersebut digunakan sebagai dasar dalam menentukan urutan pengiriman paket yang direkomendasikan oleh sistem.

Kompleksitas waktu algoritma Dijkstra adalah:

$$O((V + E) \log V)$$

dengan:

- V = jumlah vertex (node)
- E = jumlah edge

Semakin banyak node dan edge yang terdapat pada graph, semakin besar pula proses komputasi yang diperlukan. Namun, algoritma Dijkstra tetap menjadi salah satu algoritma yang efisien untuk pencarian jalur terpendek pada graph berbobot dengan bobot positif.

BAB III

ANALISIS DAN PERANCANGAN

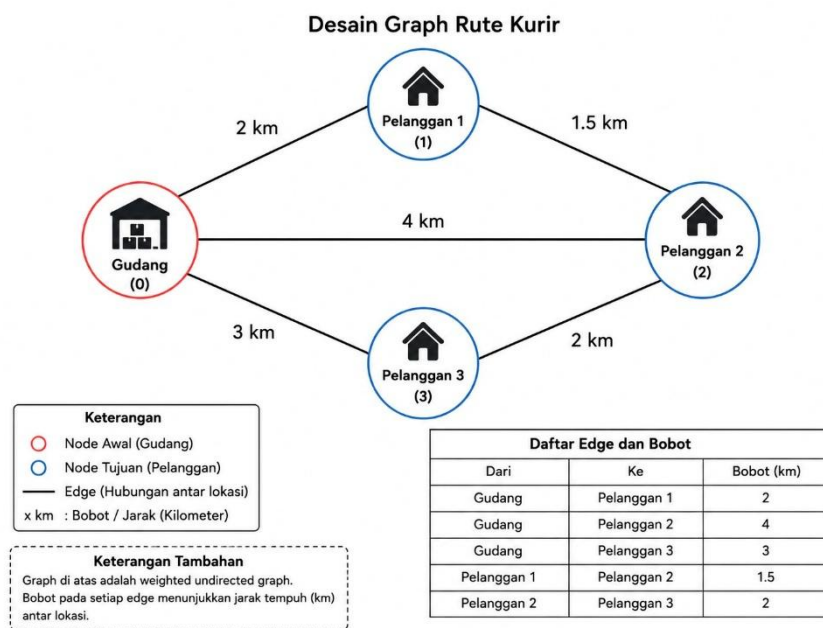
A. Analisis masalah

Dalam proses pengiriman paket, kurir sering kali harus mengunjungi beberapa lokasi pelanggan dalam satu perjalanan. Penentuan urutan pengiriman yang kurang tepat dapat menyebabkan jarak tempuh menjadi lebih jauh, waktu pengiriman bertambah lama, serta meningkatkan biaya operasional.

Pada umumnya, kurir menentukan rute berdasarkan pengalaman atau perkiraan pribadi. Cara tersebut tidak selalu menghasilkan jalur yang efisien karena tidak mempertimbangkan seluruh hubungan antar lokasi secara sistematis.

Oleh karena itu, diperlukan sebuah Sistem Pendukung Keputusan (DSS) yang mampu membantu kurir menentukan rute pengiriman yang lebih optimal. Sistem ini memanfaatkan struktur data graph untuk merepresentasikan hubungan antar lokasi dan algoritma Dijkstra untuk mencari jalur dengan jarak terpendek

B. Desain graph



Graph tersebut merupakan **Weighted Undirected Graph**, yaitu graph yang memiliki bobot pada setiap edge dan hubungan antar node dapat dilalui dua arah. Pada graph tersebut terdapat empat node yang terdiri dari satu node awal yaitu **Gudang** dan tiga node tujuan yaitu **Pelanggan 1**, **Pelanggan 2**, dan **Pelanggan 3**. Setiap node merepresentasikan lokasi yang akan dilalui dalam proses pengiriman paket.

Hubungan antar lokasi direpresentasikan oleh edge (garis penghubung). Setiap edge memiliki bobot berupa jarak tempuh dalam satuan kilometer (km). Bobot tersebut

digunakan sebagai dasar perhitungan algoritma Dijkstra untuk menentukan jalur dengan jarak terpendek.

Adapun bobot pada setiap edge adalah sebagai berikut:

Gudang – Pelanggan 1 = 2 km

Gudang – Pelanggan 2 = 4 km

Gudang – Pelanggan 3 = 3 km

Pelanggan 1 – Pelanggan 2 = 1,5 km

Pelanggan 2 – Pelanggan 3 = 2 km

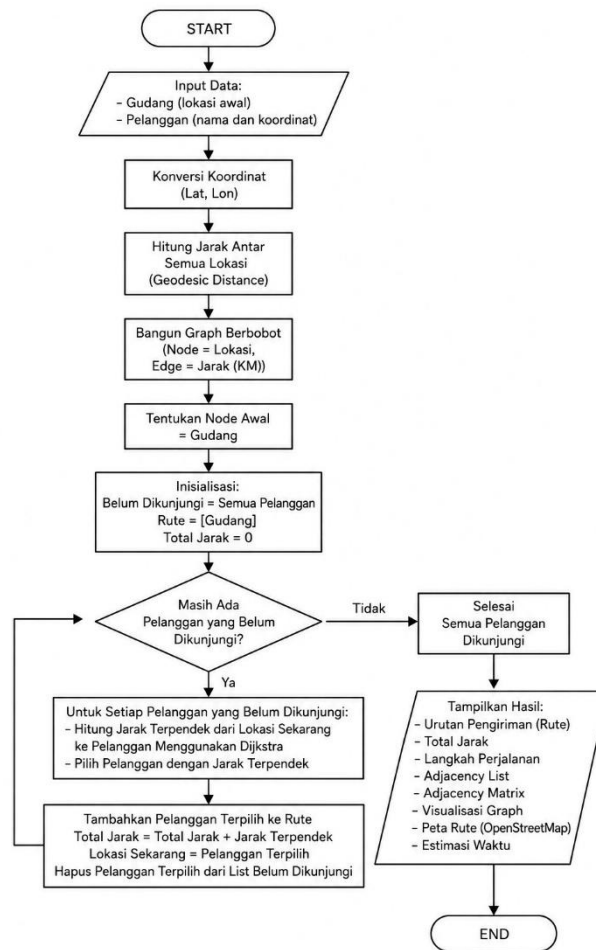
Dalam sistem yang dibangun, Gudang berfungsi sebagai node awal (source node) tempat kurir memulai perjalanan. Selanjutnya algoritma Dijkstra akan menghitung jarak terpendek menuju pelanggan yang belum dikunjungi berdasarkan bobot pada setiap edge. Hasil perhitungan tersebut digunakan untuk menghasilkan rekomendasi urutan pengiriman yang lebih efisien sehingga dapat mengurangi jarak tempuh dan waktu pengiriman.

Desain graph ini menunjukkan bagaimana hubungan antar lokasi dapat direpresentasikan dalam bentuk struktur data graph untuk mendukung proses pengambilan keputusan pada sistem penentuan rute kurir.

C. Flowchart

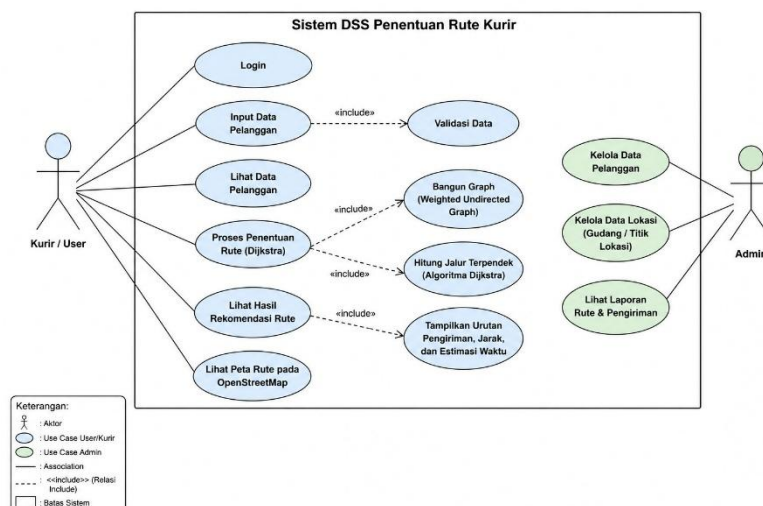
Alur kerja sistem dimulai dari proses input data pelanggan hingga menghasilkan rekomendasi rute pengiriman.

1. Pengguna memasukkan data pelanggan beserta koordinat lokasi.
2. Sistem mengonversi koordinat menjadi data lokasi.
3. Sistem menghitung jarak antar seluruh lokasi menggunakan metode Geodesic Distance.
4. Sistem membangun graph berbobot.
5. Gudang ditetapkan sebagai node awal.
6. Algoritma Dijkstra digunakan untuk mencari jalur terpendek.
7. Sistem menentukan pelanggan terdekat yang belum dikunjungi.
8. Proses diulang hingga seluruh pelanggan dikunjungi.
9. Sistem menampilkan hasil rekomendasi rute, total jarak, visualisasi graph, dan peta rute.



D. Use case diagram

Use Case Diagram digunakan untuk menggambarkan interaksi antara pengguna dengan sistem DSS Penentuan Rute Kurir. Diagram ini menunjukkan fungsi-fungsi yang dapat dijalankan oleh pengguna serta proses yang dilakukan sistem dalam menghasilkan rekomendasi rute pengiriman.



Berdasarkan Gambar 3.X, terdapat dua aktor yang berinteraksi dengan sistem, yaitu Kurir/User dan Admin. Kurir merupakan pengguna utama yang dapat melakukan input data pelanggan, melihat data pelanggan, menjalankan proses penentuan rute menggunakan algoritma Dijkstra, melihat hasil rekomendasi rute, serta melihat visualisasi rute pada OpenStreetMap. Sementara itu, Admin bertugas mengelola data pelanggan, mengelola data lokasi, dan melihat laporan hasil pengiriman.

Dalam proses penentuan rute, sistem terlebih dahulu melakukan validasi terhadap data yang dimasukkan pengguna. Selanjutnya sistem membentuk Weighted Undirected Graph berdasarkan data lokasi yang tersedia dan menjalankan algoritma Dijkstra untuk menghitung jalur terpendek. Hasil perhitungan tersebut kemudian ditampilkan dalam bentuk urutan pengiriman, total jarak tempuh, estimasi waktu perjalanan, serta visualisasi rute pada peta.

Use Case Diagram ini menunjukkan bahwa sistem berfungsi sebagai Decision Support System (DSS) yang membantu kurir dalam menentukan urutan pengiriman yang lebih efisien berdasarkan perhitungan jalur terpendek menggunakan struktur data graph dan algoritma Dijkstra.

E. Struktur node dan edge

Pada sistem DSS Penentuan Rute Kurir, struktur graph terdiri dari node (vertex) dan edge yang digunakan untuk merepresentasikan lokasi pengiriman dan hubungan antar lokasi.

1. Struktur Node

Node digunakan untuk merepresentasikan setiap lokasi yang terlibat dalam proses pengiriman paket.

Contoh node pada sistem:

Node	Keterangan
Gudang Denpasar	Lokasi awal kurir
Pelanggan 1	Tujuan pengiriman
Pelanggan 2	Tujuan pengiriman
Pelanggan 3	Tujuan pengiriman

Implementasi node pada program ditunjukkan pada kode berikut:

```
for lokasi in semua:
    G.add_node(lokasi[0])
```

Kode tersebut berfungsi untuk menambahkan seluruh lokasi ke dalam graph sebagai node.

2. Struktur Edge

Edge digunakan untuk menghubungkan node satu dengan node lainnya. Setiap edge memiliki bobot (weight) berupa jarak antar lokasi yang dihitung menggunakan metode Geodesic Distance.

Implementasi edge pada program ditunjukkan pada kode berikut:

```
jarak = geodesic((lat1, lon1), (lat2, lon2)).km

if jarak <= MAX_JARAK:
    G.add_edge(
        n1,
        n2,
        weight=round(jarak, 2)
    )
```

Parameter weight digunakan untuk menyimpan nilai jarak antar lokasi dalam satuan kilometer (KM).

Contoh struktur graph:

Misalkan terdapat data:

- Gudang Denpasar
- Pelanggan 1
- Pelanggan 2
- Pelanggan 3

Maka graph yang terbentuk dapat direpresentasikan sebagai berikut:

Gudang ↔ Pelanggan 1 = 2 km

Gudang ↔ Pelanggan 2 = 4 km

Gudang ↔ Pelanggan 3 = 3 km

Pelanggan 1 ↔ Pelanggan 2 = 1,5 km

Pelanggan 2 ↔ Pelanggan 3 = 2 km

Struktur tersebut membentuk Weighted Undirected Graph yang digunakan oleh algoritma Dijkstra untuk menentukan jalur pengiriman dengan jarak terpendek.

BAB IV

IMPLEMENTASI

A. Implementasi program

Sistem dibangun menggunakan bahasa pemrograman Python dengan framework Streamlit sebagai antarmuka pengguna. Struktur data graph diimplementasikan menggunakan library NetworkX, sedangkan perhitungan jarak antar lokasi menggunakan library Geopy. Visualisasi peta dibuat menggunakan Folium dan OpenStreetMap.

Tahapan implementasi sistem meliputi:

1. Input data pelanggan dan koordinat lokasi.
2. Pembentukan graph berdasarkan lokasi yang dimasukkan.
3. Perhitungan bobot edge menggunakan Geodesic Distance.
4. Penerapan algoritma Dijkstra untuk menentukan jalur terpendek.
5. Menampilkan hasil rekomendasi rute.
6. Menampilkan visualisasi graph dan peta rute.

B. Penjelasan kode

1. Kode Pembuatan Graph

```
semua = [GUDANG] + pelanggan

G = nx.Graph()

for lokasi in semua:
    G.add_node(lokasi[0])

MAX_JARAK = 10
```

Kode ini digunakan untuk membuat graph tidak berarah (Undirected Graph) dan menambahkan seluruh lokasi sebagai node pada graph.

2. Kode Perhitungan Jarak

```
jarak = geodesic(
    (lat1, lon1),
    (lat2, lon2)
).km
```

Kode ini digunakan untuk menghitung jarak antar dua lokasi berdasarkan koordinat latitude dan longitude menggunakan metode Geodesic Distance.

3. Kode Pembentukan Edge

```
if jarak <= MAX_JARAK:
    G.add_edge(
        n1,
        n2,
        weight=round(jarak, 2)
    )
```

Kode ini digunakan untuk menghubungkan dua node dan memberikan bobot berupa jarak tempuh dalam kilometer.

4. Kode Algoritma Dijkstra

```
for tujuan in belum:
    try:
        jarak = nx.dijkstra_path_length(
            G,
            sekarang,
            tujuan,
            weight="weight"
        )

        if jarak < jarak_min:
            jarak_min = jarak
            terdekat = tujuan

    except:
        pass

    if terdekat is None:
        break
```

Kode di atas digunakan untuk menentukan lokasi pelanggan terdekat yang belum dikunjungi. Sistem melakukan perulangan terhadap seluruh pelanggan yang masih berada dalam daftar belum. Untuk setiap pelanggan, algoritma Dijkstra digunakan untuk menghitung jarak terpendek dari lokasi saat ini (sekarang) menuju lokasi tujuan. Jika jarak yang diperoleh lebih kecil dibandingkan jarak minimum sebelumnya, maka pelanggan tersebut akan dipilih sebagai tujuan berikutnya. Apabila tidak ditemukan jalur yang dapat dilalui, proses akan dihentikan menggunakan perintah break.

```
path = nx.dijkstra_path(
    G,
    sekarang,
    terdekat,
    weight="weight"
)
```

Setelah pelanggan terdekat ditemukan, fungsi `nx.dijkstra_path()` digunakan untuk memperoleh jalur yang harus dilalui dari lokasi saat ini menuju pelanggan terpilih. Jalur tersebut kemudian ditampilkan sebagai rekomendasi rute pengiriman.

5. Kode Menampilkan Rekomendasi

```
st.write(" → ".join(path))

langkah.append(
    [sekarang, terdekat, round(jarak_min, 2)]
)

total_jarak += jarak_min
urutan.append(terdekat)

sekarang = terdekat
belum.remove(terdekat)

st.subheader("📦 Urutan Pengiriman")

rute_text = " → ".join(urutan)
st.success(rute_text)
```

Kode di atas digunakan untuk menyusun dan menampilkan hasil rekomendasi rute pengiriman. Sistem terlebih dahulu menampilkan jalur hasil perhitungan algoritma Dijkstra, kemudian menyimpan data perjalanan yang terdiri dari lokasi asal, lokasi tujuan, dan jarak tempuh. Selanjutnya sistem memperbarui total jarak perjalanan, menambahkan pelanggan yang telah dipilih ke dalam urutan pengiriman, serta menghapus pelanggan tersebut dari daftar pelanggan yang belum dikunjungi. Setelah seluruh proses selesai, sistem menampilkan rekomendasi rute akhir yang harus dilalui kurir dalam bentuk urutan lokasi pengiriman.

6. Kode Visualisasi Graph

```
nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=2500,
    node_color="lightblue",
    ax=ax
)

edge_labels = nx.get_edge_attributes(
    G,
    "weight"
)

nx.draw_networkx_edge_labels(
    G,
    pos,
    edge_labels=edge_labels,
    ax=ax
)
```

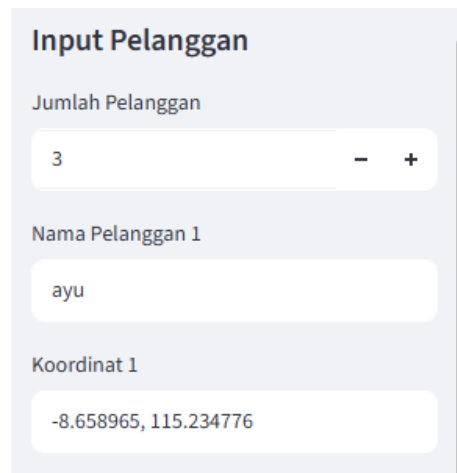
Kode di atas digunakan untuk menampilkan visualisasi graph yang telah terbentuk. Fungsi `nx.draw()` digunakan untuk menggambar node dan edge pada

graph beserta label nama lokasi. Parameter `node_size` mengatur ukuran node, sedangkan `node_color` mengatur warna node agar lebih mudah dibedakan. Selanjutnya, fungsi `nx.get_edge_attributes()` digunakan untuk mengambil nilai bobot (`weight`) dari setiap edge. Bobot tersebut kemudian ditampilkan pada graph menggunakan fungsi `nx.draw_networkx_edge_labels()`, sehingga pengguna dapat melihat jarak antar lokasi yang terhubung.

C. Tampilan sistem

1. Halaman input data

Halaman ini digunakan untuk memasukkan jumlah pelanggan, nama pelanggan, dan koordinat lokasi pelanggan.



Input Pelanggan

Jumlah Pelanggan

3 - +

Nama Pelanggan 1

ayu

Koordinat 1

-8.658965, 115.234776

2. Tampilan hasil perhitungan rute

Menunjukkan hasil perhitungan rute menggunakan algoritma Dijkstra. Sistem menampilkan jalur yang dilalui pada setiap langkah pengiriman, dimulai dari Gudang Denpasar menuju pelanggan yang memiliki jarak terdekat. Setelah seluruh pelanggan diproses, sistem menampilkan urutan pengiriman yang direkomendasikan.

Jalur Dijkstra

Gudang Denpasar → ayu

ayu → asep

asep → tika

Urutan Pengiriman

Gudang Denpasar → ayu → asep → tika

3. Tampilan langkah perjalanan dan AI Recommendation

Menunjukkan hasil perhitungan perjalanan yang dilakukan oleh sistem. Tabel Langkah Perjalanan menampilkan informasi lokasi asal, lokasi tujuan, dan jarak

Langkah Perjalanan

	Dari	Ke	Jarak (KM)
0	Gudang Denpasar	ayu	2.75
1	ayu	asep	3.87
2	asep	tika	5.86

Total Jarak

12.48 KM

Estimasi Waktu

25 Menit

AI Recommendation

Rute yang direkomendasikan adalah Gudang Denpasar → ayu → asep → tika karena pada setiap langkah sistem memilih node dengan jarak terdekat menggunakan algoritma Dijkstra.

tempuh pada setiap tahap pengiriman. Selain itu, sistem menyediakan fitur AI Recommendation yang memberikan rekomendasi urutan pengiriman berdasarkan hasil perhitungan algoritma Dijkstra.

4. Tampilan Adjacency List

Menunjukkan representasi graph dalam bentuk Adjacency List. Struktur ini menampilkan daftar node yang terhubung dengan setiap node pada graph. Representasi Adjacency List digunakan untuk menggambarkan hubungan antar node secara sederhana dan efisien. Pada sistem ini, Adjacency List membantu menunjukkan keterhubungan antar lokasi pengiriman yang nantinya digunakan dalam proses pencarian jalur menggunakan algoritma Dijkstra.

Adjacency List

```
{
  "Gudang Denpasar" : [
    0 : "ayu"
    1 : "tika"
    2 : "asep"
  ]
  "ayu" : [
    0 : "Gudang Denpasar"
    1 : "tika"
    2 : "asep"
  ]
  "tika" : [
    0 : "Gudang Denpasar"
    1 : "ayu"
    2 : "asep"
  ]
}
```


5. Tampilan Adjacency Matrix

Menunjukkan representasi graph dalam bentuk Adjacency Matrix. Adjacency Matrix merupakan tabel yang digunakan untuk menunjukkan hubungan antar node beserta bobot edge yang menghubungkannya. Baris dan kolom pada tabel merepresentasikan node, sedangkan nilai pada perpotongan baris dan kolom menunjukkan jarak antar lokasi dalam satuan kilometer (KM).

Adjacency Matrix

	Gudang Denpasar	ayu	tika	asep
Gudang Denpasar	0	2.75	3.42	3.48
ayu	2.75	0	6.14	3.87
tika	3.42	6.14	0	5.86
asep	3.48	3.87	5.86	0

6. Tampilan Statistik Graph dan Ranking Kunjungan

Menampilkan informasi statistik graph yang terbentuk dari data lokasi pengiriman. Selain itu, sistem juga menampilkan Ranking Kunjungan, yaitu urutan pelanggan yang direkomendasikan untuk dikunjungi berdasarkan hasil perhitungan algoritma Dijkstra.

Statistik Graph

Jumlah Node

4

Jumlah Edge

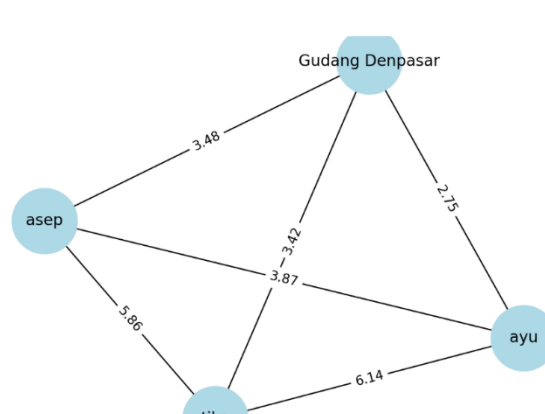
6

Ranking Kunjungan

Urutan	Pelanggan
0	1 ayu
1	2 asep
2	3 tika

7. Tampilan Visualisasi Graph

Menunjukkan visualisasi graph yang terbentuk dari data lokasi pengiriman. Setiap lingkaran pada graph merepresentasikan node, Garis yang menghubungkan antar node merupakan edge yang menunjukkan adanya hubungan antar lokasi. Angka yang ditampilkan pada setiap edge merupakan bobot (weight) berupa jarak antar lokasi dalam satuan kilometer (KM). Bobot



tersebut digunakan oleh algoritma Dijkstra untuk menentukan jalur dengan jarak terpendek.

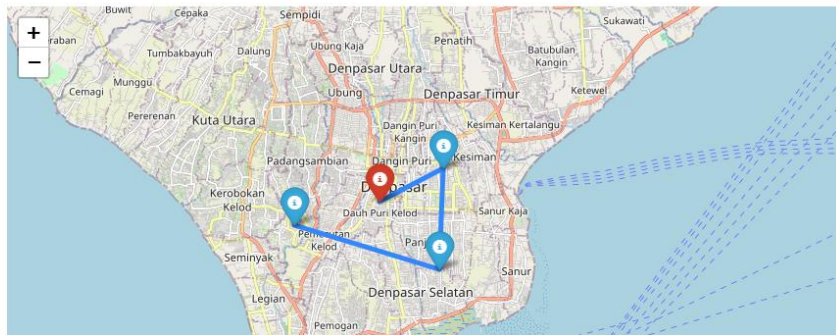
8. Tampilan Kompleksitas Algoritma dan Visualisasi Peta

Menampilkan informasi mengenai kompleksitas algoritma yang digunakan dalam sistem, yaitu algoritma Dijkstra dengan kompleksitas waktu $O((V + E) \log V)$, di mana V merupakan jumlah node (vertex) dan E merupakan jumlah edge pada graph. Informasi ini menunjukkan tingkat efisiensi algoritma dalam mencari jalur terpendek pada graph berbobot. Selain itu, sistem juga

Kompleksitas Algoritma

Dijkstra memiliki kompleksitas $O((V + E) \log V)$, di mana V adalah jumlah node dan E adalah jumlah edge.

OpenStreetMap



menampilkan visualisasi rute pengiriman menggunakan OpenStreetMap. Visualisasi peta ini membantu pengguna memahami lokasi tujuan dan jalur pengiriman secara lebih intuitif.

BAB V

PENGUJIAN DAN ANALISIS

A. Pengujian

1. Skenario pengujian

Pengujian dilakukan untuk memastikan sistem dapat membentuk graph, menjalankan algoritma Dijkstra, serta menghasilkan rekomendasi rute pengiriman yang sesuai.

No	Pengujian	Input	Hasil yang Diharapkan
1	Input data pelanggan	Nama dan koordinat pelanggan	Data berhasil tersimpan
2	Pembentukan graph	Data lokasi pelanggan	Node dan edge berhasil terbentuk
3	Perhitungan jarak	Koordinat lokasi	Bobot edge berhasil dihitung
4	Algoritma Dijkstra	Data graph	Jalur terpendek berhasil ditemukan
5	Visualisasi graph	Data graph	Graph berhasil ditampilkan
6	Visualisasi peta	Data koordinat	Peta dan rute berhasil ditampilkan

2. Data uji

Lokasi	Koordinat
Gudang Denpasar	(-8.6705, 115.2126)
ayu	(-8.6950, 115.2100)
asep	(-8.7000, 115.2450)
tika	(-8.7300, 115.2300)

3. Hasil pengujian

Dari data uji yang digunakan, berikut adalah hasil pengujian:

Langkah Perjalanan

	Dari	Ke	Jarak (KM)
0	Gudang Denpasar	ayu	2.75
1	ayu	asep	3.87
2	asep	tika	5.86

Total Jarak

12.48 KM

Estimasi Waktu

25 Menit

AI Recommendation

Rute yang direkomendasikan adalah Gudang Denpasar → ayu → asep → tika karena pada setiap langkah sistem memilih node dengan jarak terdekat menggunakan algoritma Dijkstra.

B. Analisis hasil

Berdasarkan hasil pengujian yang dilakukan, sistem berhasil membentuk graph dari data lokasi yang dimasukkan pengguna. Setiap lokasi direpresentasikan sebagai node dan

hubungan antar lokasi direpresentasikan sebagai edge yang memiliki bobot berupa jarak tempuh.

Algoritma Dijkstra berhasil menentukan urutan pengiriman berdasarkan jarak terpendek dari lokasi saat ini menuju pelanggan berikutnya. Pada data uji yang digunakan, sistem menghasilkan rekomendasi rute:

Gudang Denpasar → ayu → asepe → tika

Hasil tersebut sesuai dengan perhitungan jarak yang terdapat pada graph sehingga dapat membantu kurir menentukan urutan pengiriman yang lebih efisien.

Selain itu, sistem juga berhasil menampilkan representasi graph dalam bentuk Adjacency List, Adjacency Matrix, visualisasi graph, serta visualisasi peta menggunakan OpenStreetMap. Fitur-fitur tersebut memudahkan pengguna dalam memahami hubungan antar lokasi dan jalur pengiriman yang direkomendasikan. Secara keseluruhan, sistem telah memenuhi fungsi utama sebagai Decision Support System (DSS) untuk membantu pengambilan keputusan dalam penentuan rute kurir.

C. Kompleksitas algoritma

Sistem menggunakan algoritma Dijkstra untuk mencari jalur terpendek pada graph berbobot.

Kompleksitas waktu algoritma Dijkstra adalah:

$$O((V + E) \log V)$$

Keterangan:

- V = jumlah node (vertex)
- E = jumlah edge

Pada pengujian yang dilakukan terdapat:

- Jumlah Node (V) = 4
- Jumlah Edge (E) = 6

Dengan jumlah node dan edge yang relatif kecil, proses perhitungan dapat dilakukan dengan sangat cepat. Semakin banyak lokasi yang dimasukkan, maka jumlah node dan edge akan bertambah sehingga waktu komputasi juga meningkat. Namun, algoritma Dijkstra tetap cukup efisien untuk digunakan pada sistem penentuan rute kurir dengan jumlah lokasi pengiriman harian yang tidak terlalu besar.

BAB VI

KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem **Decision Support System (DSS) Penentuan Rute Kurir** berhasil dibangun menggunakan struktur data graph dan algoritma Dijkstra. Sistem mampu merepresentasikan lokasi pengiriman sebagai node dan hubungan antar lokasi sebagai edge yang memiliki bobot berupa jarak tempuh.

Algoritma Dijkstra berhasil digunakan untuk menentukan jalur pengiriman berdasarkan jarak terpendek sehingga dapat menghasilkan rekomendasi urutan pengiriman yang lebih efisien. Selain itu, sistem juga mampu menampilkan Adjacency List, Adjacency Matrix, visualisasi graph, serta visualisasi rute pada peta OpenStreetMap yang membantu pengguna dalam memahami hasil perhitungan yang dilakukan.

Dengan demikian, penerapan struktur data graph pada sistem ini membuktikan bahwa graph tidak hanya digunakan sebagai konsep teoritis, tetapi juga dapat dimanfaatkan sebagai dasar pengambilan keputusan pada permasalahan nyata, khususnya dalam penentuan rute pengiriman paket oleh kurir.

B. Saran pengembangan

Beberapa saran yang dapat dilakukan untuk pengembangan sistem di masa mendatang adalah sebagai berikut:

1. Menambahkan integrasi dengan API peta seperti Google Maps atau OpenRouteService untuk memperoleh rute jalan yang lebih akurat.
2. Menambahkan informasi kondisi lalu lintas secara real-time sehingga rekomendasi rute dapat disesuaikan dengan kondisi jalan saat itu.
3. Mengembangkan sistem agar dapat menangani jumlah lokasi pengiriman yang lebih banyak dan lebih kompleks.
4. Menambahkan fitur optimasi rute yang lebih lanjut menggunakan algoritma lain seperti A* atau Traveling Salesman Problem (TSP).
5. Mengembangkan sistem berbasis web atau mobile agar dapat digunakan secara langsung oleh kurir di lapangan.

C. Daftar pustaka

- Dijkstra, E. W. (1959). *A Note on Two Problems in Connexion with Graphs*. Numerische Mathematik, 1(1), 269–271.
- Zulkifli, Z., & Sariffudin, S. (2017). *Decision Support System Pemberian Bonus Tahunan Pada Karyawan Berdasarkan Kinerja Karyawan Menggunakan Metode Simple Additive Weighting*. TAM, 7, 67–73.